COMINDIR02: Particionamento e Recuperação com Publish/Subscribe

Documento de Estudo e Registo de Desenvolvimento (Sprint E1)

Sistemas Distribuídos - ESTGV

Outubro 2026

Conteúdo

1 Arquitetura e Conceitos Base **2**
- 1.1 Partições, Líderes e Réplicas . . . 2
- 1.2 O Fluxo de uma Mensagem (Exemplo com S12) . . . 2

2 Comandos e Ambiente **2**
- 2.1 IPFS e Rede (Gossip) . . . 2
- 2.2 Compilação e Execução . . . 2

3 Dissecção do Código Fonte **3**
- 3.1 `Wire.java` (O Dicionário de Rede) . . . 3
- 3.2 `RecordFile.java` e o método `range` . . . 3
- 3.3 `Producer.java` (A Entrada) . . . 3
- 3.4 `PartitionLeader.java` (O Coração) . . . 4
- 3.5 `PartitionReplica.java` (A Inteligência) . . . 4

4 Registo de Dúvidas e Respostas **5**

5 Comportamento em Caso de Falhas **5**

1 Arquitetura e Conceitos Base

O sistema introduz duas grandes novidades em relação à COMINDIR01: o **particionamento** e a **recuperação TCP**.

1.1 Partições, Líderes e Réplicas

- **Partição:** Um subconjunto de dados isolado. Por analogia, é como dividir as filas de um supermercado por 3 caixas independentes. Com $\mathbf{N = 3}$, temos as partições `p0`, `p1` e `p2`.
- **Líder:** Cada partição tem um processo líder próprio (ex: líder p0, líder p1). O líder é responsável por atribuir os números de sequência (`seq`) *da sua partição.*
- **Fator de Replicação ($\mathbf{R = 2}$):** Cada partição existe em exatamente 2 réplicas. Se uma réplica falhar, a outra garante a disponibilidade dos dados.
- **Chave de Partição:** O ID do sensor. A função `Math.floorMod(sensorId.hashCode(), n)` garante que o mesmo sensor cai sempre na mesma partição.

1.2 O Fluxo de uma Mensagem (Exemplo com S12)

Para compreender o caminho, suponhamos o envio do sensor `S12`. 1. **Produtor:** Calcula o *hash* de "S12", aplica o `floorMod` e determina que pertence à `p2`. Publica no tópico de entrada `rep02-in-p2`. 2. **Líder 2:** Ouve esse tópico, atribui o `seq=10`, grava no `lider-p2.txt` e publica a mensagem `REG` no tópico de dados `rep02-p2`. 3. **Réplicas:** As réplicas que guardam a `p2` (no caso, a Réplica 2 e a Réplica 3) ouvem o tópico de dados e gravam nos seus ficheiros (`replica-2-p2.txt` e `replica-3-p2.txt`).

2 Comandos e Ambiente

2.1 IPFS e Rede (Gossip)

Para evitar interferência de outros colegas, arrancar o daemon em modo *offline*:

```
ipfs daemon --offline
```

Para o cenário de dois nós IPFS (como no COMINDIR01), em que notámos perdas de mensagens na rede distribuída:

1. Arrancar o Nó 1 (porta 5001): `ipfs daemon`
2. Arrancar o Nó 2 num diretório separado: `IPFS_PATH=~/.ipfs2 ipfs daemon`
3. Obter o PeerID do Nó 2: `ipfs id` (no terminal do Nó 2)
4. Conectar os nós (Gossip): `ipfs swarm connect /ip4/127.0.0.1/tcp/4001/p2p/<PeerID_do_No_2>`

2.2 Compilação e Execução

Dentro de cada uma das 3 pastas (`rep02-lider`, `rep02-produtor`, `rep02-replica`):

```
javac -d out src/rep02/*.java
```

Ordem de Arranque: Réplicas → Líderes → Produtor.

```
# 1. Replicas
java -cp out rep02.PartitionReplica 1 0,1
java -cp out rep02.PartitionReplica 2 1,2
java -Dperda=0.2 -cp out rep02.PartitionReplica 3 2,0

# 2. Lideres
java -cp out rep02.PartitionLeader 0 3
java -cp out rep02.PartitionLeader 1 3
java -cp out rep02.PartitionLeader 2 3

# 3. Produtor
java -cp out rep02.Producer 3

```

3 Dissecção do Código Fonte

3.1 `Wire.java` (O Dicionário de Rede)

Define as constantes e os formatos de todas as mensagens. A função `regParts` é fulcral: usa `split(";", 4)`. Como a mensagem `REG` tem a forma `REG;IP;Porta;seq;sensor;temp;hora`, o limite `4` garante que o 4.º elemento devolvido é a linha inteira do registo (`seq;sensor;temp;hora`), pronta a ser lida pelo `SensorRecord.fromLine()`, sem termos de reconstruir os pedaços com `;`.

3.2 `RecordFile.java` e o método `range`

O método lê o ficheiro inteiro e filtra os registos entre `from` e `to`.

```
public synchronized List<SensorRecord> range(long from, long to) throws
IOException {
  List<SensorRecord> out = new ArrayList<>();
  if (!Files.exists(path)) return out;
  for (String line : Files.readAllLines(path, StandardCharsets.UTF_8)) {
    if (line.isBlank()) continue;
    SensorRecord r = SensorRecord.fromLine(line);
    if (r.getSeq() >= from && r.getSeq() <= to) out.add(r);
  }
  return out;
}

```

O `synchronized` é vital: se o líder estiver a fazer `append` na thread de entrada, a thread TCP (que chama o `range`) fica bloqueada à espera, garantindo que não lê um ficheiro a meio de uma escrita. O custo é **O(N)** (lê o ficheiro todo mesmo para 1 registo).

3.3 `Producer.java` (A Entrada)

Lê comandos da consola e encaminha para os tópicos.

- **Comando `rajada n s`:** Gera `n` leituras. O `s` é o número máximo de sensores (ex: `s=20` gera sensores S01 a S20). O tempo é medido *depois* de cada `publish`, medindo o tempo de confirmação da rede.
- **Comando `rajada-quente n s`:** 80% das leituras vão para o S01, criando um hotspot na partição 0.
- **Tratamento de Erros:** Um `try/catch` envolve a publicação. Se o IPFS estiver offline, o produtor imprime o erro mas continua a correr.

3.4 `PartitionLeader.java` (O Coração)

O líder tem duas *threads* simultâneas.

1. **Thread Secundária (`entrada`):** Fica bloqueada no `ps.subscribe()` a ler mensagens `IN`.
2. **Thread Principal (`main`):** Fica bloqueada no `server.accept()` num ciclo `while(true)` à espera de ligações TCP das réplicas.

A Secção Crítica (LOCK):

```
    synchronized (LOCK) {
      long seq = ultimoSeq + 1;
      r = SensorRecord.now(seq, sensor, temp);
      file.append(r);      // Grava no ficheiro
      ultimoSeq = seq;     // Atualiza a variavel
    }
    ps.publish(Wire.topicData(k), Wire.reg(endereco, porto, r));

```

Porque não fazer o publish dentro do LOCK? O publish é uma chamada de rede (HTTP) que pode demorar. Se fosse dentro do LOCK, bloqueava a thread TCP de recuperar os dados. Por isso, grava-se primeiro e publica-se depois (fora do LOCK).

3.5 `PartitionReplica.java` (A Inteligência)

A réplica gere o estado de cada partição através de uma classe aninhada `Particao`.

- **Thread por Partição:** O `main` lança uma *thread* para subscrever o tópico `rep02-p0`, outra para `rep02-p1`, etc. Isto garante que se a p0 estiver a recuperar (bloqueada), a p1 continua a receber dados.
- `SCHED.scheduleAtFixedRate`: Uma thread *daemon* que verifica de 200 em 200 ms se a partição esteve 1 segundo em silêncio (`SILENCIO_MS`). Se sim, imprime as métricas de fim de rajada.
- **A Regra de Ouro (método `aplicar`):**
  - Se `seq <= ultimo` ou já está no `TreeMap`: **Duplicado** (ignora).
  - Se `seq == ultimo + 1`: **Em ordem** (grava e tenta drenar os retidos).
  - Se `seq > ultimo + 1`: **Adiantado** (guarda no `TreeMap retidos`).

O Gatilho de Recuperação: Se `retidos.size() > LIMITE_RETIDOS` (10), a réplica abre um `Socket` TCP para o líder:

```
    long de = ultimo + 1;
    long ate = retidos.firstKey() - 1; // O minimo dos retidos menos 1
    // ... abre socket e envia "PEDIDO de ate" ...
    while ((linha = in.readLine()) != null) {
      if (linha.equals("FIM")) break;
      aplicar(SensorRecord.fromLine(linha)); // Aplica os recuperados!
    }

```

4 Registo de Dúvidas e Respostas

Registo de Dúvidas e Respostas

Dúvida: O que é o `floorMod` e porque não usar `%`?
Resposta: O operador `%` em Java mantém o sinal do dividendo. Se o `hashCode()` for negativo (ex: -5), `-5 % 3` dá `-2`. Isso geraria um nome de tópico inválido (ex: `rep02-in-p-2`). O `Math.floorMod(-5, 3)` devolve `1` (o resto matemático positivo), garantindo índices válidos de 0 a N-1.

Registo de Dúvidas e Respostas

Dúvida: O comando `-Dperda=0.2` não estava a funcionar.
Resposta: O `-D` define uma propriedade de sistema. Tem de ser passado à JVM *antes* do nome da classe (ex: `java -Dperda=0.2 -cp out rep02...`). No IntelliJ, deve ir no campo *VM Options*, e não em *Program Arguments*. No código, é lido via `System.getProperty("perda", "0")`.

Registo de Dúvidas e Respostas

Dúvida: Na réplica, porque usamos `out.println("FIM")` e não `System.out.println()`?
Resposta: `System.out` imprime no terminal local. O `out` é um `PrintWriter` associado ao `Socket TCP`. Portanto, o `out.println("FIM")` envia a string "FIM"pela rede para a réplica, sinalizando o fim do ficheiro.

Registo de Dúvidas e Respostas

Dúvida: O líder cria uma thread por cada ligação TCP de réplica?
Resposta: **Não.** O líder usa um ciclo `while(true) { server.accept() }` na thread principal. Ele atende uma ligação de cada vez. Se duas réplicas se ligarem em simultâneo, a segunda fica em espera. Isto é um *gargalo* conhecido (*head-of-line blocking*), discutido na reflexão da ficha.

5 Comportamento em Caso de Falhas

Cenário de Falha	**Comportamento Esperado**
Ficheiro do líder apagado	Líder reinicia `seq` para 1. As réplicas (que já têm registos maiores) ignoram-nos como duplicados, causando divergência total.
Perda do último registo da série	Invisível. A réplica não tem como detetar a falta, pois não chegam mensagens posteriores para criar o gatilho dos 10 retidos.
Líder inacessível (IP errado)	A replicação PubSub funciona, mas os pedidos de recuperação TCP falham por *Timeout* (5000ms).
Porto TCP ocupado	`BindException` no arranque. A thread de entrada continua a receber, mas perde-se a capacidade de recuperar dados por TCP.
Ligação TCP cortada sem FIM	`RECUPERAÇÃO FALHOU: resposta interrompida`. Os dados parciais já gravados mantêm-se, e a réplica tentará novamente no próximo `REG`.
Líder parado durante a rajada	As leituras para esse tópico perdem-se para sempre, pois o PubSub não guarda mensagens. O líder retoma a numeração quando volta.